Kalite Mühendisliği: Ürün Geliştirmede Temel Araçlar

ÖFY

October 2026

Özet

Kalite, ürünün müşterinin beklediği işi beklediği biçimde yapmasıdır; hatasızlık bu tanımın yalnızca bir parçasıdır. Kalite kontrol hatayı ürün çıktıktan sonra yakalar, kalite güvence ise hatanın baştan oluşmasını engeller; hata ne kadar geç bulunursa maliyeti o kadar artar. Üretim kökenli kalite araçlarının çoğu yazılıma doğrudan aktarılabilir: yalın düşüncenin israf türleri yazılım ekiplerinde birebir karşılık bulur, yedi temel kalite aracı bug ve şikâyet verisini önceliklendirir, FMEA ise bir özelliğin nasıl bozulabileceğini tasarım aşamasında puanlayarak test tasarımının girdisi olur. RPN çarpımı tek başına karar mekanizması değildir; yüksek şiddetli riskler düşük çarpım yüzünden geri plana atılmamalıdır. PUKÖ döngüsünün son adımı olan standartlaştırma, başarılı bir çözümü dokümantasyondan mimari kurala uzanan seviyelerde kalıcı mekanizmaya çevirir ve hatayı sistem aracılığıyla önleyen çözümler insan disiplinine dayananlardan daha güçlüdür. Bir çözümün tek senaryoda çalışması güvenilir olduğunu göstermez; sınır durum testleri çözümün nerede bozulabileceğini ortaya koyar. Henüz yaşanmamış hata senaryoları tahminle değil, sistemin alan modelinden, durum modelinden, bağımlılıklarından ve sınır koşullarından sistematik olarak türetilir.

Giriş

Kalite mühendisliği bilgisi tarihsel olarak imalat sanayisinde, özellikle otomotiv ve Japon üretim sisteminde gelişmiştir. Bu yüzden kalite literatürünün dili fabrika sahasının dilidir: hat, parça, tolerans, muayene. Yazılım ya da dijital ürün geliştiren biri için sorun, bu araçların hangisinin aktarılabilir olduğunu ve aktarıldığında neye karşılık geldiğini bilmektir.

Merkezî terimler şunlardır. **Kalite**, ürünün müşterinin beklediği işi beklediği biçimde yapmasıdır; kullanışlılık, hız ve güvenilirlik bu tanımın içindedir. **Kalite güvence**, hatanın oluşmasını önleyen süreçlerin bütünüdür; **kalite kontrol** ise ortaya çıkmış hatanın yakalanmasıdır. **Risk**, henüz gerçekleşmemiş bir hatanın etkisi ve olasılığıdır. **Standartlaştırma**, bir çözümün kişiye bağlı olmaktan çıkıp tekrarlanabilir bir yönteme dönüşmesidir.

Yaygın bir yanlış kabul, kalitenin test aşamasında sağlandığıdır. Test yalnızca kontrol tarafıdır; kalitenin büyük kısmı gereksinim, tasarım ve süreç aşamasında belirlenir. Bir diğer yanlış kabul de doldurulmuş bir risk tablosunun ya da geçen bir testin sistemin doğru olduğunu kanıtladığıdır; her iki araç da sistemin ne kadar iyi modellendiğiyle sınırlıdır.

Not, ürün geliştirme açısından gereken kalite kavramlarını kapsar: yazılım kalite güvencesi, yalın yönetim, yedi temel kalite aracı, FMEA, PUKÖ ve 8D, ISO 9001, ISO 27001, Lean Six Sigma ve temel proje yönetimi. Bu kavramların bir yazılım sistemine uygulanması EARS ses yönetim platformu üzerinden örneklenir. Otomotive, havacılığa ve kaynaklı imalata özgü araçlar (APQP, PPAP, MSA, SPC, GD&T ve benzerleri) kapsam dışıdır.

Yazılım Kalite Güvencesi

Kalitenin Tanımı

Kalite, ürünün müşterinin beklediği işi beklediği biçimde yapmasıdır. Yalnızca hatasız olmak yetmez; ürünün kullanışlı, hızlı ve güvenilir olması da gerekir.

QA ile QC Farkı

- **Kalite kontrol** (QC, *quality control*): ürün ortaya çıktıktan sonra hatayı yakalamaktır. Test yazmak ve test etmek bu gruba girer.
- **Kalite güvence** (QA, *quality assurance*): hatanın baştan oluşmamasını sağlamaktır. Kod inceleme kuralları, standartlar ve süreçler bu gruba girer.

Kısaca QC "hatayı bul", QA "hata çıkmasın" der.

Yazılım Kalitesinin Özellikleri

ISO/IEC 25010 standardı yazılım ürün kalitesini şu özelliklerle tanımlar (ISO/IEC 25010, 2011):

- İşlevsellik: ürün doğru işi yapıyor mu?
- Güvenilirlik: çökmeden çalışıyor mu?
- Kullanılabilirlik: kullanımı kolay mı?
- Performans: hızlı mı?
- Bakım kolaylığı: değiştirmesi kolay mı?
- Taşınabilirlik: başka ortamda çalışır mı?
- Güvenlik.

Kalite Güvence Planı

Kalite güvence planı şu soruları cevaplar:

- Hangi standartlara uyulacak?
- Hangi aşamada neye bakılacak (gereksinim, tasarım, kod, test, yayın)?
- Sorumlu kim?
- Hata çıktığında nasıl kaydedilip takip edilecek?

Ana İlke

Hata ne kadar geç bulunursa o kadar pahalıya mal olur. Gereksinim aşamasında yakalanan bir hata, canlı sistemde yakalanan hatadan kat kat ucuzdur (Boehm, 1981).

Yalın Yönetim

Yalın yönetim (*lean*) Toyota üretim sisteminden doğmuştur (Ohno, 1988). Ana fikri şudur: müşteriye değer katmayan her şey israftır ve atılmalıdır.

7+1 İsraf ve Yazılımdaki Karşılıkları

1. Fazla üretim: kimsenin kullanmadığı özellik yapmak.
2. Bekleme: onay, kod incelemesi ya da yayın beklemek.
3. Taşıma: işi ekipten ekibe gereksiz yere devretmek.
4. Fazla işleme: gerekenden karmaşık çözüm yapmak.
5. Stok: yarım kalmış işler, birleştirilmemiş dallar (*branch*).
6. Hareket: araç ve ekran arasında gidip gelmek, bilgi aramak.
7. Hata: bug'lar ve yeniden yapılan işler.
8. Kullanılmayan yetenek: ekibin fikrini almamak.

Muda, Mura, Muri

- Muda: israf.
- Mura: düzensizlik; bir hafta çok iş, bir hafta boş.
- Muri: aşırı yük; ekibi kapasitesinin üstünde zorlamak.

Kalite Fonksiyon Yayılımı ve Kalite Evi

Kalite fonksiyon yayılımı (QFD, *quality function deployment*), müşterinin isteğini teknik gereksinime çeviren bir tablodur; tablonun görünümü nedeniyle "kalite evi" olarak da anılır (Akao, 1990).

- Satırlarda müşterinin sözleri yer alır: "hızlı açılsın", "kolay olsun".
- Sütunlarda teknik ölçüler yer alır: "açılış süresi < 2 sn", "en fazla 3 tıklama".
- Hangi teknik ölçünün hangi müşteri isteğini karşıladığı işaretlenir.
- Böylece önce neyin yapılacağı belli olur.

JIT ve Jidoka

- Tam zamanında üretim (JIT, *just in time*): bir şeyi ihtiyaç olduğu anda yapmak, önceden yığmamak. Yazılımdaki karşılığı küçük ve sık yayınlardır.
- Jidoka: hata görülünce hattı durdurmak. Yazılımdaki karşılığı, CI (sürekli entegrasyon, *continuous integration*) testi kırıldığında birleştirmenin engellenmesidir.

Yedi Temel Kalite Aracı

Yedi temel kalite aracı veriyle karar vermek için kullanılan basit araçlardır ve Ishikawa tarafından bir set olarak yaygınlaştırılmıştır (Ishikawa, 1976).

Akış Şeması

Süreç kutu ve oklarla çizilir. Örnek: "Kullanıcı kayıt olur, mail gider, kullanıcı onaylar." Darboğazlar ve gereksiz adımlar görünür hâle gelir.

Kontrol Çizelgesi (Çetele)

Olayların sayılmasıdır. Örnek: "Bu hafta hangi hata türünden kaç tane geldi?"

Pareto

Sorunların yaklaşık %80'i nedenlerin yaklaşık %20'sinden çıkar. Hatalar en çoktan en aza sıralanır; ilk birkaçı çözüldüğünde sorunun büyük kısmı biter.

Histogram

Verinin dağılımını gösteren sütun grafiktir. Örnek: sayfa açılış sürelerinin çoğu 1 saniye civarında mı, yoksa bir kısmı 10 saniyeye mi uzanıyor?

Balık Kılçığı (Ishikawa, Sebep-Sonuç Diyagramı)

Balığın başında sorun, kılçıklarda olası nedenler yer alır. Kılçıklar insan, yöntem, araç, malzeme, ölçüm ve ortam diye gruplanır. Beyin fırtınası için kullanılır.

Serpilme Diyagramı

Serpilme diyagramı (*scatter plot*), iki değişken arasındaki ilişkiyi noktalar üzerinden gösteren bir grafik türüdür. İstatistikte ve veri analizinde kullanılır.

Okunuşu

Her nokta, iki değişkenin birlikte ölçüldüğü bir gözlemi temsil eder. Örneğin bir insanın günlük spor süresiyle kondisyon puanı incelendiğinde:

- X ekseni: günlük spor süresi.
- Y ekseni: kondisyon puanı.
- Her nokta: bir kişinin ölçümü.

Üç Temel Örüntü

1. Pozitif ilişki: spor süresi arttıkça kondisyon puanı da artma eğilimindedir.
2. Negatif ilişki: bir değişken artarken diğeri azalma eğilimindedir.
3. Belirgin ilişki yok: noktalar belirli bir yön ya da örüntü göstermez.

Kullanım Amaçları

- İki değişken arasında ilişki olup olmadığı görülür.
- İlişkinin yönü ve yaklaşık gücü değerlendirilir.
- Diğer gözlemlerden çok farklı olan aykırı değerler fark edilir.
- İlişkinin doğrusal mı, yoksa daha karmaşık bir örüntü mü olduğu incelenir.

İlişki ve Nedensellik

İki değişkenin ilişkili olması birinin diğerine neden olduğunu kanıtlamaz. Spor süresiyle kondisyon arasında pozitif ilişki görülse bile kondisyonu etkileyen beslenme, uyku ve sağlık gibi başka faktörler bulunabilir. Ürün bağlamında örnek: kullanıcı sayısı arttıkça hata sayısının da artıp artmadığı serpilme diyagramıyla görülebilir. Kontrol teorisinde ve sistem analizinde de iki ölçüm arasındaki ilişkiyi ilk kez keşfetmek için kullanılabilir.

Kontrol Grafiği

Bir metrik zaman içinde izlenir ve grafiğe üst ve alt sınırlar çizilir (Shewhart, 1931). Değer sınır dışına çıkarsa “olağan dışı bir şey oldu” demektir ve incelenir.

FMEA: Hata Türü ve Etkileri Analizi

FMEA (*failure mode and effects analysis*), "ne bozulabilir" sorusunu iş bozulmadan önce sormaktır. Bir sistemi yapmadan önce o sistemin hangi biçimlerde başarısız olabileceğini sistematik olarak düşünme refleksidir. Değeri yalnızca hata listesi çıkarmasında değil, hataları önem sırasına koyarak mühendislik kaynaklarının nereye harcanacağını belirlemesindedir.

Tablo Yapısı

Her özellik için bir tabloya şu sorular yazılır:

- Nasıl bozulabilir? Örnek: ödeme iki kez çekilir.
- Etkisi ne olur? Müşteri parasını kaybeder, şikâyet eder.
- Nedeni ne olabilir? Kullanıcı butona iki kez basar.
- Şu an nasıl önleniyor ya da yakalanıyor?

Puanlama

Her satıra 1 ile 10 arasında üç puan verilir:

- Şiddet (S): etkisi ne kadar kötü?
- Olasılık (O): ne kadar sık olur?
- Tespit (D): müşteriden önce ne kadar kolay yakalanır? Puan yüksekse hata zor yakalanıyor demektir.

Risk öncelik sayısı (RPN, *risk priority number*) üç puanın çarpımıdır:

$$RPN = S \times O \times D$$

Yüksek çıkan satırlardan başlanarak önlem alınır.

EARS Örneği: Ses Kaydı İndirme

EARS, yayın kuruluşları için geliştirilmiş bir ses yönetim platformudur. Kullanıcı bir ses kaydını indirdiğinde şu FMEA satırı yazılabilir:

- Hata: kullanıcı aynı kaydı iki kez indirir.
- Etkisi: indirme sayacı yanlış artabilir ya da kullanıcı aynı dosyayı tekrar tekrar alabilir.
- Olası neden: çift tıklama, tekrarlanan HTTP isteği ya da yeniden deneme mekanizması.

- Mevcut kontrol: istek başına mı kayıt tutuluyor, yoksa gerçekten tamamlanan indirme mi sayılıyor?
- Önlem: işlem kimliği kullanılarak tekrarlar engellenir ya da indirme sayacının hangi olayı ölçtüğü netleştirilir.

FMEA'nın burada yaptırdığı şey, hata ortaya çıktıktan sonra bug düzeltmek yerine bu ihtimalin tasarım sırasında düşünülmesidir.

Puanlama Örneği

Aşağıdaki sayılar tamamen örnektir:

Hata	S	O	D	RPN
Sayaç iki kez artıyor	4	6	5	120
Yanlış ses kaydı indiriliyor	9	3	8	216
Filtre sonucu boş görünüyor	3	5	2	30

Yanlış ses kaydının indirilmesi daha az sıklıkla yaşansa bile, etkisi ağır ve tespiti zor olduğu için daha yüksek öncelik alır.

RPN'nin Sınırı ve Eylem Önceliği

RPN tek başına karar mekanizması değildir; farklı riskler aynı çarpımı verebilir. Örneğin $S = 10, O = 2, D = 3$ ile $S = 3, O = 5, D = 4$ satırlarının ikisi de 60 verir, ama birincisi ağır sonuçlu bir risktir. 2019'da yayımlanan AIAG-VDA FMEA el kitabı, çarpım yerine eylem önceliği (AP, *action priority*) tablosunu kullanır; bu tablo şiddet, oluşma ve tespit puanlarının kombinasyonuna göre aksiyon önceliğini belirler ve özellikle şiddeti yüksek riskleri öne çıkarır (AIAG & VDA, 2019). Yüksek şiddetli riskler yalnızca çarpımları düşük çıktığı için göz ardı edilmemelidir.

FMEA ile Kalite Güvence Arasındaki Bağ

- FMEA (risk analizi): sistemde ne bozulabilir ve hangi hatanın sonucu daha ağır olur?
- Test tasarımı: belirlenen riskler hangi testlerle ortaya çıkarılabilir?
- Kontrol ve doğrulama: hatanın oluşması nasıl engellenir, oluşursa nasıl yakalanır?

Bir ses indirme sistemi için FMEA'dan şu testler çıkabilir:

- Aynı indirme isteği art arda gönderilir.
- İstek sürerken ağ bağlantısı kesilir.
- Kullanıcının yetkisi olmayan bir kaydı indirmesi denenir.

- Aynı cut numarasına sahip ama farklı `libloc` değerleri olan kayıtlar test edilir.
- Backend yeniden başladığında yarım kalmış işlemler kontrol edilir.

Bunlar rastgele testler değil, tanımlanmış risklerden türetilmiş testlerdir.

FMEA'nın Arkasındaki Düşünce ve Sınırları

Bir sistemi test etmek için önce onun nasıl başarısız olabileceğine dair bir model kurulur. Bu düşünce sistem mühendisliği ve kontrol düşüncesiyle bağlantılıdır: sistemin yalnızca normal davranışı değil, hata durumları, sınırları ve hata karşısındaki tepkisi de modellenir.

FMEA her şeyi çözmez. Bilinmeyen bir hata türünü kendiliğinden bulamaz; analizin kalitesi sistemin ne kadar iyi anlaşıldığına bağlıdır. Doldurulmuş bir FMEA tablosu sistemin güvenli olduğunu kanıtlamaz. Kalite güvence açısından FMEA, hangi testlerin yazılacağına karar vermeden önce yapılan risk odaklı düşünme çalışmasıdır; test otomasyonu ise o riskleri kontrol etmenin araçlarından biridir. Yazılımdaki genel karşılığı, yeni bir özellik tasarlanırken yapılan risk oturumudur.

Problem Çözme: PUKÖ ve 8D

PUKÖ (PDCA)

PUKÖ (Planla, Uygula, Kontrol et, Önlem al; İngilizcesi PDCA, *plan-do-check-act*) sürekli iyileştirmenin döngüsüdür. Shewhart'ın fikri üzerine Deming tarafından yaygınlaştırılmıştır (Deming, 1986).

1. Planla: sorun ve hedef belirlenir, çözüm fikri üretilir.
2. Uygula: çözüm küçük ölçekte denenir.
3. Kontrol et: sonucun beklendiği gibi olup olmadığı ölçülür.
4. Önlem al: işe yaradıysa standart yapılır, yaramadıysa başa dönülür.

Yazılımdaki karşılığı A/B testi ve sprint döngüsüdür.

"İşe Yaradıysa Standart Yap" İfadesinin Anlamı

Standartlaştırma, başarılı bir çözümü kişilerin hafızasına ya da iyi niyetine bırakmayıp işin kalıcı bir parçası hâline getirmektir.

Örnek: Backend'in Yanlış Process'e Bağlanması

Backend yeniden başlatıldığında eski process 8000 portunda çalışmaya devam ediyorsa, geliştirici yeni kodu test ettiğini sanarken aslında eski kod cevap verir.

- Planla: testten önce 8000 portundaki process'in doğru process olduğu doğrulanır.
- Uygula: bu kontrol birkaç geliştirme oturumunda denenir.
- Kontrol et: yanlış process yüzünden eski kodun test edilmesi sorunu ortadan kalktı mı?
- Önlem al: işe yaradıysa kontrol kalıcı geliştirme prosedürüne eklenir.

Son adımda şunlar değişir:

- README ya da geliştirme dokümantasyonuna kontrol adımı eklenir.
- Mümkünse kontrol bir script ile otomatikleştirilir.
- Geliştiriciler aynı kontrolü her seferinde uygular.
- Sorun tekrar ortaya çıkarsa ekip nereden başlayacağını bilir.

Böylece çözüm, bir kişinin o gün yaptığı bir hareket olmaktan çıkar ve ekibin tekrarlanabilir çalışma yöntemi olur.

Standartlaştırmanın Seviyeleri

Standartlaştırma her zaman dokümantasyon yazmak demek değildir:

Seviye	Örnek
Dokümantasyon	README'ye doğru başlatma adımlarını yazmak
Prosedür	PR öncesi belirli kontrolleri zorunlu kılmak
Otomasyon	Yanlış yapılandırmayı tespit eden script yazmak
Mimari kural	Hatalı yapılandırmanın oluşmasını tasarım gereği engellemek
Ölçüm	Sorunun tekrar edip etmediğini takip etmek

Bu seviyeler içinde, hatayı yalnızca insanlara hatırlatmak yerine sistem aracılığıyla önlemek daha güçlü bir çözümdür. Geliştiricilere "testleri çalıştırmayı unutmayın" demek bir yöntemdir; CI pipeline'ında testler geçmeden birleştirmeyi engellemek ise aynı kuralı teknik olarak uygulamaktır. Bir çözüm, gerçekten işe yaradığı ve farklı durumlarda da güvenilir olduğu doğrulanmadan standartlaştırılmamalıdır.

PUKÖ'nün son adımı, bir sorunu bir kez çözmekten çıkıp aynı sorunun tekrar oluşma ihtimalini azaltan kalıcı bir mekanizma kurmaktır.

Farklı Durumlarda Güvenilirlik: Aynı Cut Numarası Örneği

Bir çözümün tek bir senaryoda çalışması genel olarak doğru olduğu anlamına gelmez. EARS'ta iki farklı kütüphanede aynı cut numarası bulunabilir:

- Kayıt A: `cut=00042`, `libloc=C:`, `libname=ENCOLIB`
- Kayıt B: `cut=00042`, `libloc=D:`, `libname=NEWS`

Sistem ses dosyasını yalnızca cut numarasına göre bulursa yanlış kaydı indirebilir.

Planla

Dosya seçiminde `libloc` ve `libname` bilgilerinin de kullanılmasına karar verilir.

Uygula

Değişiklik yapılır ve iki kaydın aynı cut numarasına sahip olduğu bir test çalıştırılır. Doğru dosya indirilir; ancak bu, çözümün güvenilir olduğunu henüz göstermez.

Kontrol Et: Farklı Durumların Denenmesi

- Durum A – Normal çalışma: iki farklı kütüphanede aynı cut numarası vardır; doğru dosya bulunur.

- Durum B – Aynı numara, üç farklı kütüphane: sistem yine doğru kaydı ayırt ediyor mu?
- Durum C – `libname` eksik: sistem yanlış kaydı mı seçiyor, yoksa belirsizliği fark edip hata mı veriyor?
- Durum D – Aynı kayıt için eşzamanlı iki istek: istekler çakıştığında dosya eşlemesi ya da indirme takibi bozuluyor mu?
- Durum E – DLE bağlantısı kesiliyor: sistem tekrar bağlandığında aynı kayıt doğru biçimde bulunabiliyor mu?

Bunlar çözümün sınırlarını araştıran testlerdir. Hepsinin hata vereceği iddia edilmez; amaç, çözümün farklı koşullarda da doğru kaldığını doğrulamaktır.

Önlem Al: Çözümün Standartlaştırılması

Testler başarılıysa:

- `libloc` ve `libname` ile eşleştirme kuralı kalıcı hâle getirilir.
- Normal ve sınır durumları otomatik testlere eklenir.
- İleride biri kodu değiştirip yalnızca cut numarasını kullanmaya dönerse testler bunu yakalar.

Mühendislik açısından fark şudur: ilk test çözümün çalıştığını gösterir; farklı koşullardaki testler çözümün nerede çalıştığını ve nerede bozulabileceğini gösterir. Otomatik regresyon testleri bu bilginin sonraki değişikliklerde korunmasını sağlar.

8D

8D (*eight disciplines*), büyük ve tekrarlayan sorunlar için sekiz adımlı bir problem çözme yöntemidir.

1. D1 – Ekip kur: konuyu bilen insanlar toplanır.
2. D2 – Sorunu tanımla: ne, nerede, ne zaman, ne kadar; net yazılır.
3. D3 – Geçici önlem: müşteri hemen korunur. Örnek: hotfix ya da özelliği kapatmak.
4. D4 – Kök nedeni bul: 5 Neden yöntemi ya da balık kılçığı kullanılır. Sorunun neden oluştuğu ve neden yakalanmadığı ayrı ayrı bulunur.
5. D5 – Kalıcı çözümü seç.
6. D6 – Kalıcı çözümü uygula ve işe yaradığını doğrula.
7. D7 – Tekrarını önle: süreç, test ya da kontrol listesi güncellenir.
8. D8 – Ekibi takdir et ve kapat.

5 Neden Örneği

- Site çöktü. Neden? Veritabanı doldu.
- Neden? Loglar silinmiyordu.
- Neden? Temizleme görevi yoktu.
- Neden? Kurulum listesinde böyle bir madde yoktu.
- Asıl çözüm: kurulum listesini düzeltmek.

A3

A3, aynı mantığın tek bir A3 kâğıda sığdırılmış hâlidir. Sol tarafta sorun ve analiz, sağ tarafta çözüm ve takip yer alır.

Hata Senaryolarının Sistematik Türetilmesi

Aynı cut numarasına sahip iki kayıt gibi bir senaryonun akla nasıl gelebileceği sorusu mühendisliğin temel meselelerinden biridir. İyi mühendisler hata senaryolarını yalnızca yaratıcı tahminle bulmaz; sistemin yapısından hareketle sistematik olarak türetir. Ontoloji bu iş için kullanılabilir, ancak tek başına yeterli değildir.

Sistemin Modellenmesi

EARS örneğinde sistemde şu kavramlar bulunur:

- Cut: ses kaydı numarası, örneğin `00042`.
- Kütüphane (*library*): `libloc` ve `libname` ile tanımlanır.
- Fiziksel ses dosyası: indirilecek gerçek içerik.

Bu model kurulduğunda kritik bir soru belirir: bir cut numarası sistem genelinde mi benzersizdir, yoksa yalnızca kendi kütüphanesi içinde mi? Benzersizlik yalnızca kütüphane içinde geçerliyse `cut=00042` tek başına bir kaydı tanımlamaya yetmez. Dolayısıyla aynı numaranın iki kütüphanede bulunması, sistem modelinden türetilen doğal bir test senaryosudur. Ontoloji burada kavramları, ilişkileri ve kimlik kurallarını netleştirmeye yardım eder.

Senaryo Türetmenin Tamamlayıcı Yaklaşımları

Ontoloji ve Alan Modelleme

Sistemde ne tür şeyler var, aralarındaki ilişkiler neler, hangi özellikler benzersiz, hangi ilişkiler zorunlu? Bulduğu risk türü: bir kimlik alanının tek başına yeterli olmaması.

Durum Modelleme

Sistem hangi durumlarda bulunabilir ve bir durumdan diğerine nasıl geçer? Bulduğu risk türü: indirme sürerken bağlantının kopması, isteğin tekrarlanması ya da işlemin yarım kalması.

Bağımlılık ve Arayüz Analizi

Hangi bileşen hangi veriye güveniyor; veriyi kim üretiyor, kim tüketiyor, hangi dönüşümlerden geçiriyor? Bulduğu risk türü: DLE'nin bildiği cut numarasıyla EARS'ın iç kimliğinin karıştırılması.

Sınır ve Kombinasyon Analizi

Bir alan eksikse, boşsa, tekrarlanıyorsa, geçersizse ya da başka bir alanla çelişiyorsa ne olur? Bulduğu risk türü: `libname` alanının eksik olması ya da aynı cut numarasının birden fazla yerde bulunması.

Bu yaklaşımlar farklı araçlardır: ontoloji alanı anlamlandırır, durum modeli davranışı, bağımlılık analizi veri akışını, sınır analizi ise hata koşullarını araştırır.

Modelden Test Türetme Süreci

1. Alan modeli kurulur: hangi varlıklar ve kimlikler var?
2. Kurallar tanımlanır: örneğin bir kayıt hangi alanlarla benzersiz tanımlanıyor?
3. Kuralları bozabilecek durumlar üretilir: alan eksikliği, çakışan kimlikler, geçersiz ilişkiler, tekrarlanan işlemler.
4. Test ve risk analizi yapılır: hangi ihlal yanlış sonuç üretir, hangisi kritiktir?

Bu yaklaşımın yazılım mühendisliğinde somut karşılıkları vardır: alan güdümlü tasarım (*domain-driven design*; Evans, 2003), model tabanlı test (*model-based testing*), özellik tabanlı test (*property-based testing*; Claessen ve Hughes, 2000) ve risk tabanlı test (*risk-based testing*).

Özellik tabanlı testte tek bir `00042` örneğine bağlı kalınmaz; farklı kütüphanelerde aynı cut numarasına sahip çok sayıda kayıt otomatik üretilerek sistemin her zaman doğru kaydı seçtiği sınanır. Testin doğru olabilmesi için önce doğru kuralın bilinmesi gerekir: sistemin benzersizlik kuralı yanlış modellenmişse otomatik test de yanlış varsayımı doğrular.

Ontolojinin Sınırı

Ontoloji sistemde neyin bulunduğunu anlatır; tek başına sistemin bütün hatalarını söylemez. Hata senaryolarının kaynakları veri modelindeki eksikliklerden geniştir:

- Yanlış alan varsayımı.
- Bileşenler arası sözleşme uyuşmazlığı.
- Zamanlama ve eşzamanlılık.
- Ağ ve altyapı arızaları.
- Yetki ihlalleri.
- İnsanların sistemi beklenmedik biçimde kullanması.
- Gereksinimin kendisinin yanlış olması.

Bu nedenle gerçek mühendislikte farklı modeller bir araya getirilir. Aranan yetenek yalnızca test yazmak değil, bir sistemin modelinden, kurallarından ve bağımlılıklarından henüz yaşanmamış hata durumlarını çıkarabilmektir.

Önerilen Sıra

1. Alan modelleme ve ontoloji: sistemdeki kavramlar ve ilişkiler anlaşılır.
2. Sistem modelleme: sistemin yapısı ve davranışı modellenir.
3. Hata analizi (FMEA): nelerin nasıl başarısız olabileceği araştırılır.
4. Model tabanlı ve özellik tabanlı test: modelden sistematik testler üretilir.
5. Geri bildirim: üretimde görülen hatalar modele geri eklenir.

Son adım önemlidir: model bir kez kurulup bırakılmaz, gerçek sistemden öğrenildikçe geliştirilir. Ontoloji bu sürecin bir parçasıdır; ontoloji, davranış modeli, hata analizi ve sistematik test üretimi birlikte bu problemi çok daha güçlü biçimde ele alır.

ISO 9001 Kalite Yönetim Sistemi

ISO 9001, bir şirketin kaliteyi düzenli ve sistemli yönettiğini gösteren uluslararası standarttır (ISO 9001, 2015). Kurumsal müşteriler tedarikçilerinden çoğu zaman bu belgeyi ister.

Yedi İlke

1. Müşteri odaklılık.
2. Liderlik.
3. Çalışanların katılımı.
4. Süreç yaklaşımı.
5. İyileştirme.
6. Kanıta dayalı karar verme.
7. İlişki yönetimi (tedarikçi ve paydaşlarla).

Maddeler 4–10

- Madde 4 – Bağlam: şirketi etkileyen iç ve dış konular ile müşteri, yatırımcı ve devlet gibi ilgili tarafların beklentileri belirlenir. SWOT analizi burada yapılır.
- Madde 5 – Liderlik: üst yönetim kaliteye sahip çıkar, kalite politikasını yazar, sorumlulukları dağıtır.
- Madde 6 – Planlama: riskler ve fırsatlar belirlenir, ölçülebilir kalite hedefleri konur.
- Madde 7 – Destek: kaynak, yetkin personel, eğitim, iletişim ve dokümanların yönetimi.
- Madde 8 – Operasyon: ürünün gerçekten üretildiği kısım; müşteri gereksinimleri, tasarım ve geliştirme, tedarikçi kontrolü, uygun olmayan ürünün yönetimi.
- Madde 9 – Performans: ölçme, müşteri memnuniyeti, iç tetkik (iç denetim), yönetimin gözden geçirmesi.
- Madde 10 – İyileştirme: uygunsuzluk çıktığında düzeltici faaliyet (DÖF) ve sürekli iyileştirme.

Risk Tabanlı Düşünme

Risk tabanlı düşünme, sorun çıktıktan sonra düzeltmek yerine olası sorunu önceden görüp önlem almaktır.

ISO 27001 ve İç Denetim

ISO/IEC 27001, bilgi güvenliği yönetim sistemi standardıdır (ISO/IEC 27001, 2022). Müşteri verisi tutan bir ürün kurumsal satışlarda bu soruyla karşılaşır.

Üç Temel Hedef (CIA)

- Gizlilik (*confidentiality*): veriyi yalnızca yetkili kişi görür.
- Bütünlük (*integrity*): veri izinsiz değişmez.
- Erişilebilirlik (*availability*): veri gerektiğinde ulaşılabilir durumdadır.

İç Denetimin Adımları

1. Plan: hangi süreç, ne zaman, kimin tarafından denetlenecek?
2. Denetim: dokümanlara bakılır, insanlarla konuşulur, kayıtlar incelenir. Her bulgu kanıta dayanmalıdır. Örnek: "Ayrılan çalışanların erişimi kapatılıyor mu?" sorusu için kayıt istenir.
3. Bulgular ikiye ayrılır: uygunsuzluk (kurala uyulmuyor) ve gözlem (risk var ama henüz kural ihlali yok).
4. Rapor yazılır.
5. Düzeltici faaliyet: sorumlu ve tarih belirlenir, ardından kapanıp kapanmadığı takip edilir.

Lean Six Sigma ve DMAIC

Six Sigma, süreçteki değişkenliği azaltıp hatayı en aza indirmeyi amaçlayan bir yöntemdir; hedefi milyonda 3,4 hatadır. Motorola'da geliştirilmiştir.

Kuşaklar

- Beyaz kuşak: farkındalık.
- Sarı kuşak: projede yardımcı.
- Yeşil kuşak: proje yürütür.
- Kara kuşak: büyük projeleri ve ekipleri yönetir.

DMAIC

1. Define (Tanımla): sorun ve müşterinin istediği belirlenir; müşterinin sesi (VOC, *voice of the customer*).
2. Measure (Ölç): mevcut durum ölçülür. Örnek: ödeme sayfasında terk oranı %40.
3. Analyze (Analiz et): veriyle kök neden bulunur.
4. Improve (İyileştir): çözüm denenir.
5. Control (Kontrol et): iyileşmenin kalıcı olması sağlanır; metrik izlenir, standart yazılır.

Kano Modeli

Kano modeli ürün özelliklerini üçe ayırır (Kano vd., 1984):

- Temel beklenti: olmazsa müşteri kızar, olunca kimse fark etmez.
- Performans özelliği: ne kadar iyi olursa o kadar memnun eder.
- Heyecan verici özellik: beklenmez, olunca müşteri çok sevinir.

Temel Proje Yönetimi

Proje, başı ve sonu belli, benzersiz bir iştir. **Operasyon** ise sürekli tekrarlanan iştir.

Kısıtlar Üçgeni

Kapsam, zaman ve maliyet birbirine bağlıdır; biri değiştiğinde diğerleri etkilenir. Kalite bu üçünün ortasında durur.

Temel Araçlar

- *Project charter* (başlatma belgesi): projenin amacı, sponsoru ve sınırları.
- Paydaş analizi: kimin etkilendiği, kimin sözünün geçtiği.
- WBS (iş kırılım yapısı, *work breakdown structure*): büyük işi küçük ve yönetilebilir parçalara bölmek.
- Kritik yol: gecikirse bütün projeyi geciktiren işler zinciri.
- *Scope creep*: kapsamın kontrolsüzce büyümesi. *Gold plating*: istenmeyen ekstraların eklenmesi. İkisinden de kaçınılır.
- Kazanılmış değer analizi: planlanan, harcanan ve tamamlanan iş karşılaştırılır; projenin bütçe ve takvim açısından nerede olduğu sayıyla görülür.

Projede Kalite

1. Kalite planlama: hangi standarda göre çalışılacağı.
2. Kalite güvence: sürecin doğru işleyip işlemediği.
3. Kalite kontrol: çıktının doğru olup olmadığı.

Sonuç

Üretim kökenli kalite araçlarının yazılıma aktarılabilir olanları, sektöre değil her sistemde bulunan olgulara dayananlardır: risk (FMEA), sorun (PUKÖ, 8D, 5 Neden), israf (yalın) ve veri (yedi araç). Otomotive ya da imalata özgü araçların aksine bunlar yazılım ekibinde doğrudan karşılık bulur; bekleme israfı onay kuyruğuna, Jidoka kırılan CI testine, kontrol grafiği metrik izlemeye denk düşer.

Bu araçların yazılımdaki asıl değeri test tasarımında ortaya çıkar. FMEA hangi testlerin yazılacağına karar vermeden önceki risk düşüncesidir; puanlama, sık ama hafif hataların değil, nadir ama ağır ve zor fark edilen hataların öncelik almasını sağlar. PUKÖ'nün standartlaştırma adımı ise bir çözümü dokümantasyondan mimari kurala uzanan seviyelerde kalıcı hâle getirir. Bu seviyelerde yukarı çıkıldıkça çözüm insan disiplininden bağımsızlaşır ve güvenilirliği artar.

Çerçevenin sınırı şudur: FMEA da modelden test türetme de sistemin ne kadar doğru modellendiğine bağlıdır. Yanlış kurulmuş bir benzersizlik kuralı, doldurulmuş bir FMEA tablosunu da otomatik testleri de yanıltır; testler yanlış varsayımı doğrulamış olur. Bu yüzden açık kalan soru, alan modelinin doğruluğunun nasıl sınanacağıdır. Önerilen cevap, üretimde görülen hataların sürekli olarak modele geri beslenmesidir. Bu bağ, kalite mühendisliğini yazılım mimarisi ve bilgi modelleme (ontoloji) alanlarıyla doğrudan ilişkilendirir.

ISO 9001 ve ISO 27001 ise ürünün kendisinden çok ürünü üreten kurumun güvenilirliğini belgeler. Kurumsal satışta bu belgeler teknik kaliteden bağımsız bir eşik işlevi görür: ürün iyi olsa bile süreç kanıtlanamıyorsa satış takılabilir.

Kaynaklar

- AIAG & VDA (2019). *FMEA Handbook: Design FMEA, Process FMEA, Supplemental FMEA for Monitoring and System Response*. Automotive Industry Action Group / Verband der Automobilindustrie.
- Akao, Y. (1990). *Quality Function Deployment: Integrating Customer Requirements into Product Design*. Productivity Press.
- Boehm, B. W. (1981). *Software Engineering Economics*. Prentice Hall.
- Claessen, K. ve Hughes, J. (2000). QuickCheck: A Lightweight Tool for Random Testing of Haskell Programs. *Proceedings of ICFP 2000*.
- Deming, W. E. (1986). *Out of the Crisis*. MIT Press.
- Evans, E. (2003). *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley.
- Ishikawa, K. (1976). *Guide to Quality Control*. Asian Productivity Organization.

- ISO 9001:2015. *Quality management systems – Requirements.* International Organization for Standardization.
- ISO/IEC 25010:2011. *Systems and software engineering – Systems and software Quality Requirements and Evaluation (SQuaRE) – System and software quality models.* ISO/IEC.
- ISO/IEC 27001:2022. *Information security, cybersecurity and privacy protection – Information security management systems – Requirements.* ISO/IEC.
- Kano, N., Seraku, N., Takahashi, F. ve Tsuji, S. (1984). Attractive Quality and Must-Be Quality. *Journal of the Japanese Society for Quality Control*, 14(2).
- Ohno, T. (1988). *Toyota Production System: Beyond Large-Scale Production.* Productivity Press.
- Shewhart, W. A. (1931). *Economic Control of Quality of Manufactured Product.* Van Nostrand.